1. Trang bìa & Thông tin nhóm

ĐẠI HỌC PHENIKAA

BÁO CÁO MÔN HỌC
AN TOÀN PHẦN MỀM

Nội dung thực hiện: Kiểm thử mô hình tồn kho badminton bằng JSON, sanitizer và concolic

Tên đề tài đăng ký: .....

Nhóm: ..... Lớp: .....

Thành viên / MSSV: .....

.....

Giảng viên: ThS. Vũ Quang Dũng

Hà Nội, tháng 10 năm 2026

Cập nhật bằng chứng ngày 20/10/2026. Phạm vi tổng hợp Ch1–Ch4; tạm gác Ch5–Ch6. Thông tin nhóm và phân công để trống theo yêu cầu. Báo cáo không tự xác nhận đáp ứng mọi điều kiện nộp của giảng viên.

Lưu ý tài liệu: hướng dẫn đề tài ghi CSE703093; mẫu báo cáo ghi CSE703153. Mã học phần và tên đề tài chính thức cần nhóm xác nhận trước khi nộp.

2. Tổng quan đề tài

Bài toán là giữ hàng, hoàn khi hủy và tất toán tồn kho của một sản phẩm cầu lông, có thể kèm một biến thể. Dự án môn học chạy độc lập trong Badminton_Stock_Security; ba file Java tham chiếu được đóng băng để truy vết quy tắc nghiệp vụ từ backend.

Luồng thực hiện

Nguồn Java tham chiếu + hash SHA-256 → stock_spec.json → generate_simulator.py → nghiệp vụ C. Parser stock_json_adapter.c được viết riêng và ghép thành stock_json_seclab.c. Runner Python đưa fixture JSON vào C và so sánh trạng thái cuối với oracle độc lập.

Phạm vi đầu vào

JSON chứa sáu trường gốc bắt buộc và danh sách items có 1–100 record. Mỗi quantity thuộc 1–100, tổng không quá 100. Parser kiểm tra int32 chính xác, khóa thiếu/lạ/trùng và sai kiểu; đầu vào tối đa 16.384 byte, token số tối đa 64 byte. Đây là giới hạn adapter kiểm thử, chưa phải giới hạn HTTP của website.

Fixture chứa trạng thái kho ban đầu để kiểm thử. Nó không phải request cho phép người mua chỉnh tồn kho. Action 0–15 là mã fixture; không thay thế enum trạng thái API.

Những phần đã và chưa thực hiện

Đã có đặc tả, truy vết yêu cầu, bộ sinh nghiệp vụ C, parser, kiểm thử oracle, gcov, ASan/UBSan và guard thuần số nguyên cho SMT. Không kiểm thử trực tiếp website, transaction, đồng thời, cơ sở dữ liệu, giá, voucher hoặc PayOS.

Guard chỉ kiểm tra điều kiện trả về 0–5 trước cập nhật trạng thái. Giá trị 0 không chứng minh đã giữ hàng; độ phủ 100% của guard không đại diện toàn bộ parser hoặc website.

Tài liệu đề tài cho phép kết quả không tìm thấy lỗi nếu giải thích bằng chứng và giới hạn. Mẫu báo cáo đồng thời yêu cầu before/after. Báo cáo giữ cấu trúc mẫu và ghi rõ không có cặp phiên bản lỗi/vá đã được xác nhận.

3. Phương pháp luận

Bước 1. Xác định hợp đồng

Đóng băng nguồn Java, kiểm tra fingerprint, mô tả kiểu, miền giá trị, trạng thái và yêu cầu trong đặc tả/Excel.

Bước 2. Sinh và kiểm tra C

Sinh nghiệp vụ từ rules; ghép adapter viết riêng. Kiểm tra sinh mã xác định, thay đổi đặc tả làm đổi hành vi C và từ chối đặc tả không an toàn.

Bước 3. Kiểm thử động và oracle

Chạy 41 ca hồi quy JSON, 40 ca qua DLL, black-box và coverage-guided ở seed 1, 7, 42. So sánh mã và trạng thái cuối; phân biệt crash, timeout, vi phạm bất biến, lệch oracle.

Bước 4. Kiểm tra bộ nhớ và số học

Biên dịch Clang riêng với ASan+UBSan, chạy hồi quy và đột biến có seed. Test đối chứng xác nhận công cụ bắt được lỗi mẫu; không tính lỗi mẫu là lỗi simulator.

Bước 5. Concolic và chạy lại test

Nạp guard vào SecLabFramework, nhận 6 test từ 60 input theo log người dùng. Xuất gói characterization rồi chạy lại bằng pytest trong .venv riêng.

Công cụ và ngân sách

GCC/gcov portable cho fuzz và coverage; LLVM-MinGW cho ASan+UBSan; Python/pytest cho điều phối; SecLabFramework cho concolic. Lượt JSON dài giới hạn 500 input mỗi chế độ/seed, 120 giây mỗi lượt. Coverage-guided không đồng nghĩa SMT white-box.

```
-std=c11 -Wall -Wextra -Werror -O1 -g
-fsanitize=address,undefined -fno-sanitize-recover=all
-fno-omit-frame-pointer
```

LeakSanitizer không chạy trên môi trường Windows này. Đã bổ sung Dr. Memory 2.6.0 ở chế độ leaks_only với đối chứng có/không free. GCC -fanalyzer và Clang AST/CFG đã chạy; chưa có BMC với completeness threshold. Không dùng kết quả hữu hạn để chứng minh mọi đầu vào.

4. Phân tích lỗ hổng & Bản vá

Các mục dưới là phân tích nguy cơ và cơ chế phòng vệ đã kiểm thử. Không có bản v0 lỗi được xác nhận của mục tiêu nên chưa ghi nhận CWE nào đã được vá theo cặp before/after. Không tự gán CVE.

CWE-190: tràn số nguyên

1. Vị trí & mô tả

stock_json_seclab.c, reserve dòng 20–28 và stock_reserve_guard dòng 290–297.

2. Nguyên nhân gốc rễ

Cộng int32 trước khi kiểm tra giới hạn có thể gây tràn. Guard hiện chặn quantity trước, rồi so reserved với INT_MAX - quantity; reserve dùng int64_t cho biểu thức trung gian.

3. Bằng chứng trước

Chưa có log v0 bị tràn của simulator. Log UBSan từ chương trình lỗi mẫu chỉ là đối chứng hoạt động của công cụ.

4. Cách vá / cơ chế hiện tại

Cơ chế hiện tại: kiểm tra miền quantity 1–100 trước phép trừ; từ chối tràn giữ chỗ tổng hoặc biến thể. Đây là phân tích mã đang có, không tạo bản v0 giả.

5. Bằng chứng sau

TC-SMT3 trả 3 và TC-SMT5 trả 5; 6 test guard PASS. Test tương đương guard/reserve và ASan+UBSan bổ sung bằng chứng hữu hạn.

Tham chiếu phân loại: https://cwe.mitre.org/data/definitions/190.html

Mã hiện tại, stock_json_seclab.c dòng 291–295; không phải đoạn mã v1 đối chiếu với v0.

```
if (((quantity < 1) || (quantity > 100))) return 1;

if ((available < quantity)) return 2;

if ((reserved > (2147483647 - quantity))) return 3;

if (((variant != 0) && (variant_available < quantity))) return 4;

if (((variant != 0) && (variant_reserved > (2147483647 - quantity)))) return 5;
```

CWE-787: ghi ngoài biên

1. Vị trí & mô tả

stock_json_seclab.c, parser và jsonEvaluate dòng 259–269.

2. Nguyên nhân gốc rễ

Số record và độ dài dữ liệu phải phù hợp vùng cấp phát. Parser hiện giới hạn 100 record, 16.384 byte đầu vào và 64 byte/token số.

3. Bằng chứng trước

Chưa có bản v0 hoặc log ghi ngoài biên của mục tiêu.

4. Cách vá / cơ chế hiện tại

Cơ chế hiện tại: calloc 100 phần tử int32_t; kiểm tra biên và từ chối JSON không hợp lệ. Không tuyên bố đã thay một phiên bản không kiểm tra biên.

5. Bằng chứng sau

41 hồi quy JSON, đối chiếu DLL và chiến dịch ASan đã chạy không ghi nhận lỗi trong ngân sách. Không phải chứng minh mọi chuỗi đều an toàn.

Tham chiếu phân loại: https://cwe.mitre.org/data/definitions/787.html

CWE-476: giải tham chiếu NULL

1. Vị trí & mô tả

stock_json_seclab.c, jsonEvaluate dòng 260–266.

2. Nguyên nhân gốc rễ

Cấp phát có thể thất bại; dùng con trỏ ngay sau calloc mà không kiểm tra sẽ tạo nguy cơ NULL.

3. Bằng chứng trước

Chưa có log NULL dereference của mục tiêu trước sửa.

4. Cách vá / cơ chế hiện tại

Cơ chế hiện tại: từ chối con trỏ đầu vào NULL và trả 105 khi calloc thất bại, trước khi parser sử dụng mảng.

5. Bằng chứng sau

test_json_allocation_failure nằm trong lượt pytest đạt. Đây là kiểm tra đường lỗi cấp phát, không phải bằng chứng đã vá v0.

Tham chiếu phân loại: https://cwe.mitre.org/data/definitions/476.html

CWE-401: không giải phóng bộ nhớ

1. Vị trí & mô tả

stock_json_seclab.c, jsonEvaluate dòng 263–268.

2. Nguyên nhân gốc rễ

Sau khi calloc thành công, các đường parse lỗi và nghiệp vụ phải đi qua free. Trả về sớm sau cấp phát có thể bỏ qua giải phóng.

3. Bằng chứng trước

Chưa có bằng chứng leak của mục tiêu ở v0; LeakSanitizer chưa chạy.

4. Cách vá / cơ chế hiện tại

Cơ chế hiện tại: thu mã lỗi rồi free trước khi return. Có test cân bằng cấp phát riêng.

5. Bằng chứng sau

Test cân bằng cấp phát đạt. Dr. Memory kiểm tra 141 fixture lặp 10 vòng (1.410 lần gọi), không báo definite application leak; còn potential possible leak 24 byte ở ntdll loader/TLS, giống đối chứng sạch. Không tuyên bố mọi cảnh báo đã được giải quyết.

Tham chiếu phân loại: https://cwe.mitre.org/data/definitions/401.html

Mã hiện tại, stock_json_seclab.c dòng 263–268.

```
fixture.quantities = calloc(STOCK_JSON_MAX_RECORDS, sizeof(int32_t));
if (!fixture.quantities) return 105;
int code = jsonObject(&reader, &fixture) ? jsonQuantity(&fixture) : 100;
if (!code) code = jsonStock(fixture.input, output);
free(fixture.quantities);
return code;
```

5. Bảng tổng hợp before/after

Bảng ghi đúng tình trạng bằng chứng. Không có dòng nào được đánh dấu đã vá v0 khi chưa có phiên bản lỗi và phép chạy trước/sau tương ứng.

CWE / vị trí	Trạng thái v0	Trạng thái hiện tại	Công cụ xác minh
190 / reserve, guard	Chưa có log v0 mục tiêu	Chặn nhánh tràn tổng/biến thể; 6 guard test đạt	pytest; oracle; UBSan
787 / parser	Chưa có log v0 mục tiêu	Có kiểm tra biên; không ghi nhận ASan finding trong lượt đã chạy	Hồi quy JSON; ASan
476 / calloc	Chưa có log v0 mục tiêu	Kiểm tra NULL; test tiêm thất bại cấp phát đạt	pytest
401 / jsonEvaluate	Không có v0 bị leak được xác nhận	1.410 lần gọi: 0 definite leak; còn potential possible 24 byte hệ thống	Dr. Memory + đối chứng; bộ đếm

Kết luận đối với mẫu báo cáo

Có bằng chứng kiểm thử cơ chế phòng vệ của mô hình hiện tại. Chưa có chuỗi bằng chứng tái hiện lỗi mục tiêu → bản vá → kiểm tra lại. Nếu giảng viên yêu cầu tuyệt đối bảng bản lỗi/bản vá, mục này còn thiếu và phải được xử lý theo phạm vi đề tài đã đăng ký.

Những chương trình lỗi mẫu ASan/UBSan được dùng để kiểm tra công cụ, tách khỏi mã nộp và không thể thay cho lịch sử v0/v1 của mô hình tồn kho.

6. Bằng chứng kiểm chứng

6.1. Chiến dịch JSON dài, mốc mã nguồn lịch sử

Nguồn: results/stock-json-20261018-083428-95703d48/summary.json. Sáu lượt, tổng 3.000 input; kèm 41 hồi quy và 40 ca DLL. Mẫu số coverage thuộc snapshot trong lượt này, không áp cho mã mới.

Chế độ	Seed	Input	Giây	Dòng	Nhánh
black-box	1	500	42.97	172/213	196/320
coverage-guided	1	500	39.40	185/213	225/320
black-box	7	500	40.02	172/213	200/320
coverage-guided	7	500	40.38	203/213	253/320
black-box	42	500	38.78	172/213	197/320
coverage-guided	42	500	40.37	190/213	237/320

Tất cả lượt trên ghi 0 crash, 0 timeout, 0 vi phạm bất biến và 0 lệch oracle. Coverage-guided có độ phủ cao hơn black-box ở các seed này, nhưng chưa phải phép so sánh với SMT.

6.2. ASan+UBSan dài

Seed	Input	Giây	Sanitizer finding
1	423	120.21	0
7	461	120.19	0
42	500	114.51	0

Nguồn: results/sanitizer-20261018-084125-187616c4/summary.json. Tổng 1.384 input và 41 hồi quy. Seed 1 và 7 dừng theo ngân sách thời gian; seed 42 đủ 500. Không đo coverage, không bật LeakSanitizer.

6.3. Kiểm tra tại mốc mã mới hơn

results/report-final/pytest-combined.log ghi: 34 passed, 10549 subtests passed in 12.87s. XML ghi 10.583 trong trường tests; không diễn giải thành 10.583 hàm test độc lập. Lượt chạy gồm bộ kiểm tra generator và 6 test guard.

Lượt JSON mới results/stock-json-20261018-093532-9d53d681: 100 black-box + 100 coverage-guided, seed 1; 41 hồi quy, 40 ca DLL. Độ phủ lần lượt 172/220 dòng, 187/336 nhánh và 162/220 dòng, 183/336 nhánh. Không ghi nhận bất thường.

Lượt sanitizer mới results/sanitizer-20261018-093549-a35b9b72: 100 input, seed 1, 41 hồi quy, 0 sanitizer finding. Đây là phép chạy ngắn kiểm tra mốc mới; không cộng với lượt dài để báo một chiến dịch duy nhất.

6.4. Concolic và test xuất từ GUI

Ảnh GUI Ch3 và gói xuất mới lưu tại results/ch123-20261018-202400-9a09dbce/: 60 input → giữ 6 test, 0 crash, 22/22 probe (100%) riêng guard. Ngân sách nhập 500 không có nghĩa đã chạy 500 input. Không có fallback trong log lượt này.

Test	available	reserved	quantity	v / va / vr	Trả
1	0	0	0	0 / 0 / 0	1
2	0	0	1	0 / 0 / 0	2
3	1	2147483647	1	0 / 0 / 0	3
4	1	2147483646	1	1 / 0 / 0	4
5	1	2147483646	1	1 / 1 / 2147483647	5
6	1	2147483646	1	0 / 0 / 2147483647	0

v = variant; va/vr = variant_available/variant_reserved. ch3-export-pytest.log/XML ghi 6/6 PASS trong 1,65 giây. Mã trả còn được đối chiếu oracle độc lập. TC-SMT1 trong Excel giữ quantity=101 của lượt cũ; bản xuất mới dùng quantity=0, cùng nhánh trả 1 nhưng không phải cùng fixture. Không thay dữ liệu cũ bằng kết quả thực thi giả.

6.5. Ch1: mô hình Kripke và CTL

stock_kripke.json có 5 trạng thái, khởi tạo ready: ready, reserved, rejected, cancelled, settled. Mô hình vòng đời trừu tượng, không mô tả số lượng hay tranh chấp transaction. GUI người dùng xác nhận các kết quả sau; ch1-model-check.json kiểm tra độc lập cùng đồ thị.

Thuộc tính	Kết quả tại ready	Ý nghĩa
EF(reserved)	Đúng	Có đường đi đến giữ hàng
EF(cancelled)	Đúng	Có đường đi đến hủy
EF(settled)	Đúng	Có đường đi đến tất toán
AG(!(cancelled & settled))	Đúng	Không trạng thái reachable nào mang cả hai nhãn

Kiểm tra đồ thị độc lập không có deadlock; đối chứng bỏ cạnh reserved → cancelled làm cancelled không còn tới được. Không coi các trạng thái thỏa công thức in trong console là một trace tuần tự. Loại trừ nhãn cancelled/settled không chứng minh loại trừ hai luồng backend.

6.6. Ch2: kiểu, size, record và cấp phát

Dữ liệu	Size / căn chỉnh	Cách kiểm tra
int32_t / uint32_t	4 byte	Số nguyên 32-bit; không ép chuỗi sai kiểu
Input / Stock	24 / 4 byte	Các trường 4 byte ở offset 0,4,8,12,16,20
JsonReader	24 / 8 byte	data=0, size=8, pos=16
JsonFixture	40 / 8 byte	input=0, quantities=24, count=32
Heap quantities	100 × 4 = 400 byte	calloc; vùng riêng, không nằm trong 40 byte fixture

Số liệu ABI Windows x64 lấy từ ch2-record-layouts.log. JSON là dữ liệu văn bản, không phải bản ghi nhị phân sizeof(struct). Mảng items có 1–100 record, mỗi record chỉ có quantity 1–100; tổng ≤100, đầu vào ≤16.384 byte, token số ≤64 byte. Parser kiểm tra giới hạn trước khi ghi, kiểm tra calloc NULL và free trước return.

GUI Struct Alignment không nhận typedef struct ẩn danh: đây là giới hạn nhận diện, không phải mã không có struct. GUI quét báo nghi leak tại calloc dòng 263; kiểm tra mã thấy free dòng 267. Vì vậy ghi là cảnh báo cần xác minh, không phải lỗi đã xác nhận. GCC -fanalyzer không phát cảnh báo (exit 0); điều đó không chứng minh không có lỗi.

Bốn nhóm kiểm tra Ch2 đạt: tiêm thất bại cấp phát trả 105; cân bằng cấp phát/giải phóng; 41 hồi quy CLI + 40 ca DLL; đối chứng ASan use-after-free và UBSan overflow. Lượt sanitizer ch123 chạy 41 hồi quy +100 mutation seed 1 trong 41,63 giây, 0 crash/timeout/lệch oracle/finding. Chế độ này không đo leak hoặc coverage.

6.7. Leak: công cụ riêng và đối chứng

Phép chạy	Tải thử	Definite leak	Exit
Đối chứng bỏ free	malloc 123 byte	1 leak / 123 byte	23
Đối chứng có free	Cùng chương trình	0	0
Simulator	141 fixture × 10 vòng = 1.410 lần gọi	0	0

141 fixture =41 hồi quy +100 mutation seed 1; lặp trong một tiến trình để kiểm tra tích lũy. Cả 1.410 đầu ra khớp oracle về mã và trạng thái. Đây không phải 1.410 dữ liệu fuzz độc lập. Lượt này không có ASan hoặc coverage.

Cả đối chứng sạch và simulator còn potential possible leak 24 byte có cùng stack ntdll loader/TLS. Log potential_errors được giữ, chưa kết luận triệt để là false positive. Bộ lọc/suppression hệ thống mặc định vẫn bật, không thêm suppression riêng. Kết luận chỉ là không phát hiện definite application leak với tải đã thử, không phải "không leak tuyệt đối".

Nguồn: results/leak-20261018-225714-fe111d0d/leak-summary.json; simulator-logs/DrMemory-leak_batch.exe.22444.000/results.txt và potential_errors. Dr. Memory 2.6.0 chạy leaks_only/check_leaks/count_leaks, không phải LeakSanitizer. Nếu giảng viên bắt buộc đúng LSan thì yêu cầu đó vẫn chưa thực hiện.

6.8. Ch3: AST, CFG và SMT

Phép đo	Kết quả	Phạm vi
GUI AST heuristic	McCabe 162; nesting 3; ~206 câu lệnh	Ước lượng toàn file, không riêng guard
GUI CFG guard	18 block /22 cạnh; McCabe ≈6	Ban đầu wrap phủ 9/11 block (81%)
Clang CFG guard	16 block /23 cạnh; McCabe 9	Có ENTRY/EXIT và short-circuit
AST → SMT độc lập	6 mã 0–5 SAT, replay C khớp	Miền int32; guard thuần số
Ràng buộc sau guard	UNSAT cho tràn reserved+q	Không phải BMC toàn chương trình

CFG GUI và Clang có quy tắc tách block khác nhau, không so hai chỉ số như cùng một phép đo. 100% probe (22/22) của lượt concolic cũng không đồng nhất 81% block của wrap ban đầu. GUI SAT với available=1, r=2147483647, q=1 tìm nhánh bị từ chối; thêm $r \le 2147483647 - q$ và $r + q > 2147483647$ cho UNSAT. Công thức GUI được nhập thủ công, không tự động chứng minh toàn mã C.

6.9. So sánh cùng nhánh black-box / SMT

Mục tiêu stock_reserve_guard:return 3, tức nhánh từ chối tràn reserved, không phải crash. Cùng DLL và miền: available/reserved/va/vr □[0,2147483647], quantity □[1,100], variant □{0,1}. Năm seed: 1,7,42,101,2026. Tối đa 50.000 lần gọi C mỗi phương pháp/seed; SMT timeout 5 giây.

Phương pháp	Seed	Tìm thấy	Gọi C	Giây
uniform	1	Không	50000	0.970665
edge-enriched	1	Có	32	0.000606
SMT trực tiếp	1	Có	1	0.011428
uniform	7	Không	50000	0.836751
edge-enriched	7	Có	2	0.000046
SMT trực tiếp	7	Có	1	0.011717
uniform	42	Không	50000	0.830013
edge-enriched	42	Có	4	0.000123
SMT trực tiếp	42	Có	1	0.008638
uniform	101	Không	50000	0.816968
edge-enriched	101	Có	16	0.000219
SMT trực tiếp	101	Có	1	0.009606
uniform	2026	Không	50000	0.999179
edge-enriched	2026	Có	21	0.000257
SMT trực tiếp	2026	Có	1	0.010051

Uniform: 0/5 trong ngân sách; edge-enriched: 5/5, lần gọi 32/2/4/16/21; SMT: 5/5, một lần gọi C mỗi seed. Edge-enriched chọn biên với xác suất 20% mỗi trường, còn lại uniform. SMT giải trực tiếp ràng buộc đường đi, không phải toàn bộ bộ máy concolic.

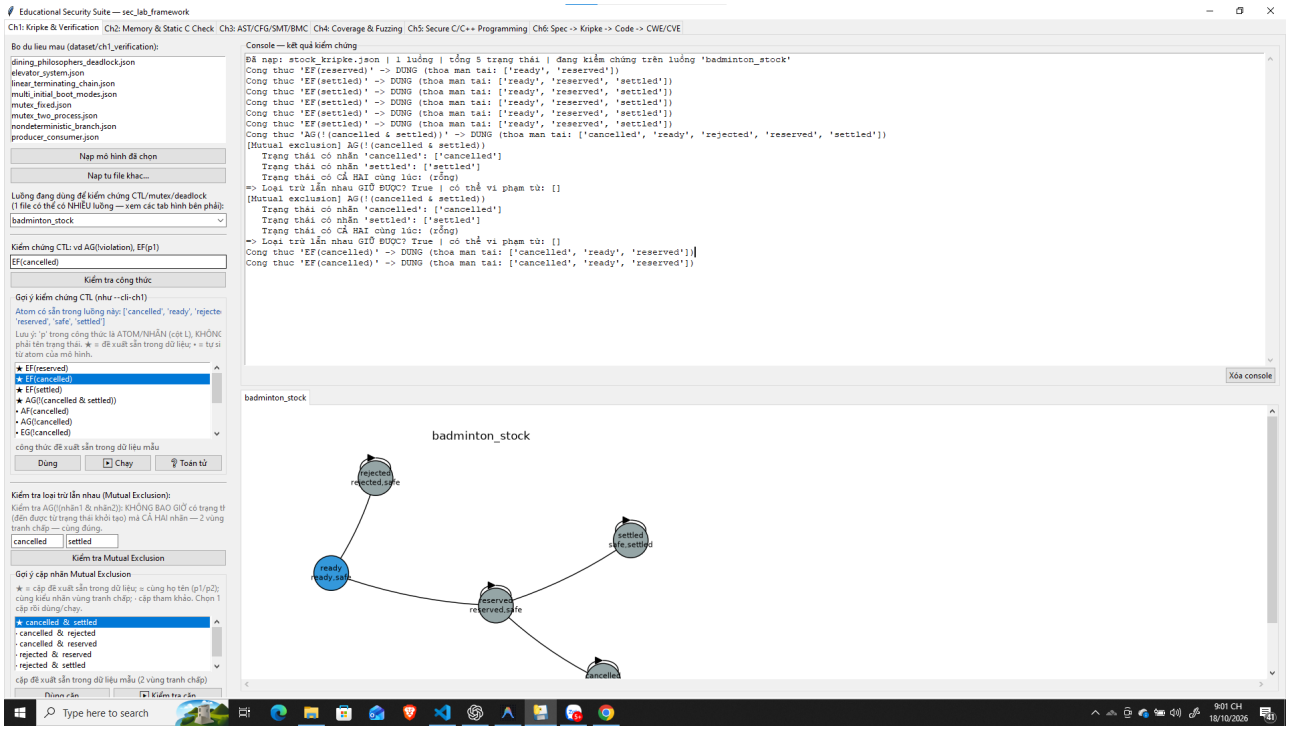
Thời gian không gồm biên dịch, nạp Z3 và đối chứng; SMT gồm tạo ràng buộc, solve và gọi C. Edge-enriched nhanh hơn SMT về thời gian ở phép thử này; SMT cần ít lần gọi C hơn. Không tính tỷ lệ tăng tốc từ các lượt uniform chưa tìm thấy, không kết luận SMT luôn nhanh hơn.

Nguồn: results/comparison-20261018-110108-14dd28b3/summary.json. pytest.xml/log cùng thư mục ghi 38 test và 10.555 subtest đạt trong 12,45 giây; không cộng subtest thành số hàm test. Lượt xuất GUI 6/6 là phép chạy riêng.

6.10. Ảnh chụp minh chứng

Hình 6.1. Ch1: CTL và loại trừ nhãn

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):

```
Cong thuc 'EF(settled)' -> DUNG (thoa man tai: ['ready', 'reserved', 'settled'])
Cong thuc 'AG(!(cancelled & settled))' -> DUNG (thoa man tai: ['cancelled', 'ready', 'rejected', 'reserved', 'settled'])
[Mutual exclusion] AG(!(cancelled & settled))
   Trạng thái có nhãn 'cancelled': ['cancelled']
   Trạng thái có nhãn 'settled': ['settled']
   Trạng thái có CẢ HAI cùng lúc: (rỗng)
=> Loại trừ lẫn nhau GIỮ ĐƯỢC? True | có thể vi phạm từ: []
[Mutual exclusion] AG(!(cancelled & settled))
   Trạng thái có nhãn 'cancelled': ['cancelled']
   Trạng thái có nhãn 'settled': ['settled']
   Trạng thái có CẢ HAI cùng lúc: (rỗng)
=> Loại trừ lẫn nhau GIỮ ĐƯỢC? True | có thể vi phạm từ: []
Cong thuc 'EF(cancelled)' -> DUNG (thoa man tai: ['cancelled', 'ready', 'reserved'])
Cong thuc 'EF(cancelled)' -> DUNG (thoa man tai: ['cancelled', 'ready', 'reserved'])
```

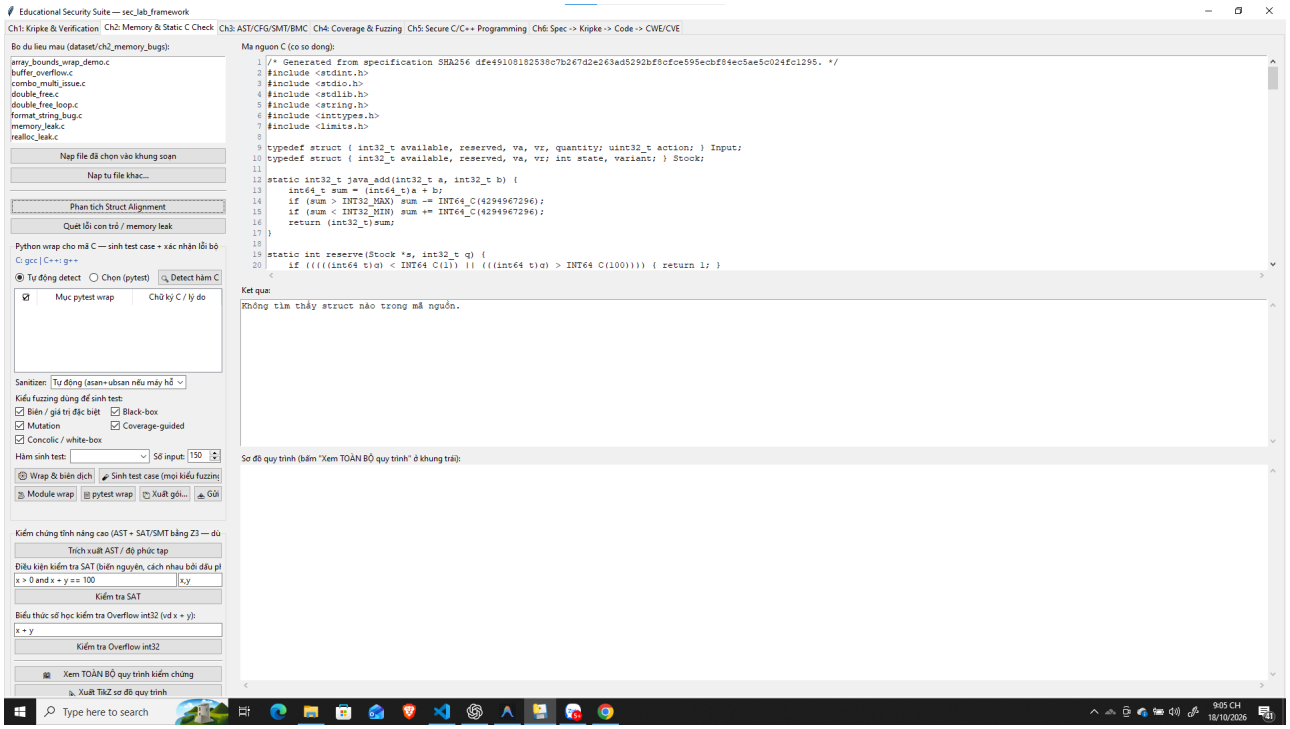
Đối chiếu mục 6.5: console ghi CTL đúng và cancelled/settled không cùng xuất hiện. Danh sách trạng thái thỏa không phải trace tuần tự; không suy ra an toàn đồng thời của backend.

Nguồn ảnh: results/report-final/ch1-gui-user-20261018.png

6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.2. Ch2: giới hạn nhận diện struct

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):



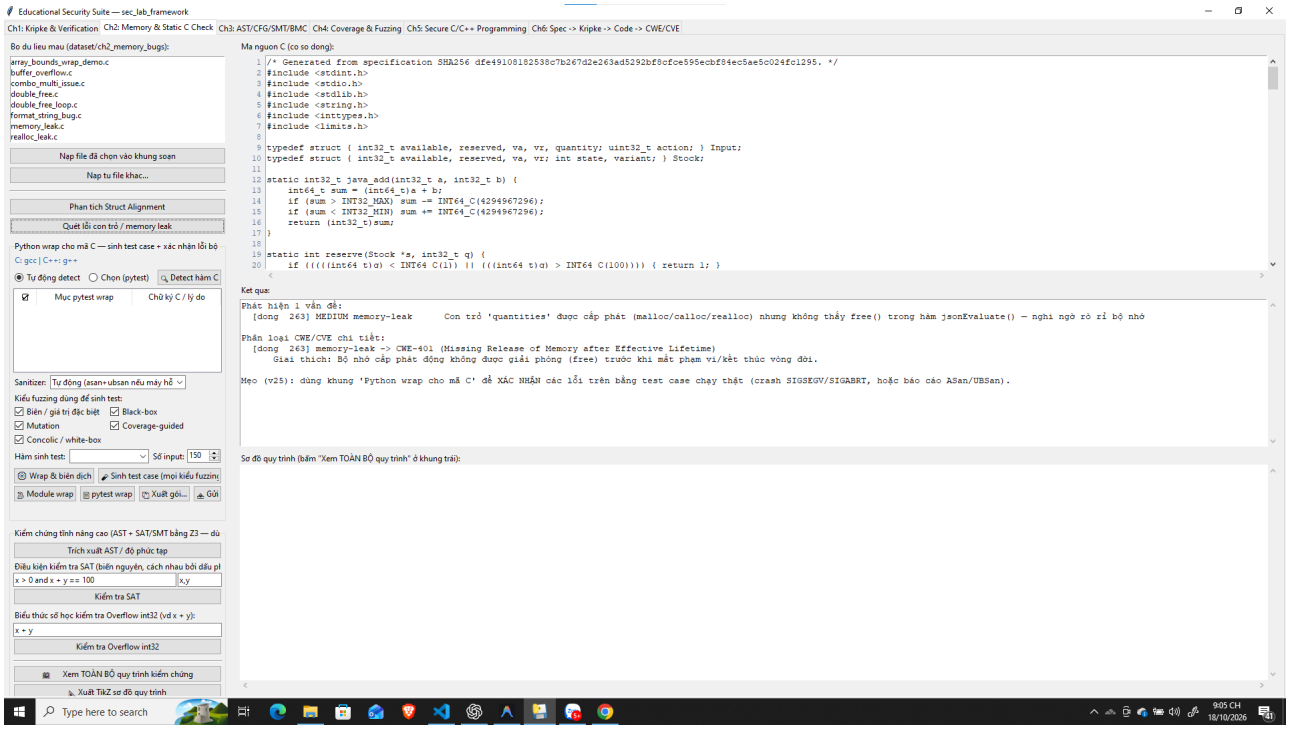
Đối chiếu mục 6.6: ứng dụng báo không tìm thấy struct dù mã có typedef struct. Đây là giới hạn công cụ; size/căn chỉnh được xác minh bằng compiler, không ghi kết quả này là PASS phân tích alignment.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch2-gui-struct.png

6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.3. Ch2: cảnh báo memory-leak cần xác minh

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):

```
Phát hiện 1 vấn đề:
  [dong  263] MEDIUM memory-leak      Con trỏ 'quantities' được cấp phát (malloc/calloc/realloc) nhưng không thấy free() trong hàm jsonEvaluate() — nghi ngờ rò rỉ bộ nhớ

Phân loại CWE/CVE chi tiết:
  [dong  263] memory-leak -> CWE-401 (Missing Release of Memory after Effective Lifetime)
      Giai thich: Bộ nhớ cấp phát động không được giải phóng (free) trước khi mất phạm vi/kết thúc vòng đời.

Mẹo (v25): dùng khung 'Python wrap cho mã C' để XÁC NHẬN các lỗi trên bằng test case chạy thật (crash SIGSEGV/SIGABRT, hoặc báo cáo ASan/UBSan).
```

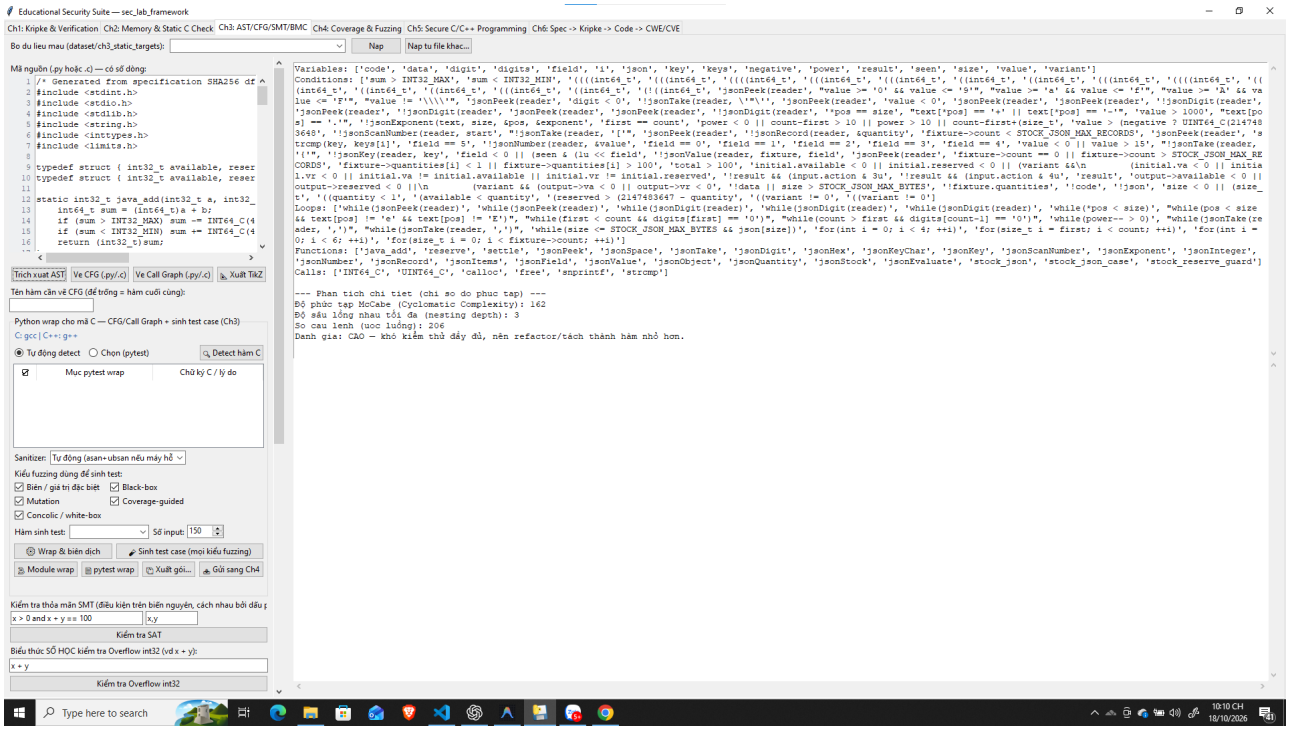
Đối chiếu mục 6.6 và CWE-401: cảnh báo nghi leak ở calloc dòng 263. Kiểm tra nguồn có free dòng 267; cảnh báo tĩnh này không phải leak sản phẩm đã được xác nhận. Bằng chứng Dr. Memory được phân tích riêng ở mục 6.7.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch2-gui-memory-warning.png

6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.4. Ch3: trích xuất AST và độ phức tạp

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):

```
--- Phan tich chi tiet (chi so do phuc tap) ---
Độ phức tạp McCabe (Cyclomatic Complexity): 162
Độ sâu lồng nhau tối đa (nesting depth): 3
So cau lenh (uoc luồng): 206
Danh gia: CAO — khó kiểm thử đầy đủ, nên refactor/tách thành hàm nhỏ hơn.
```

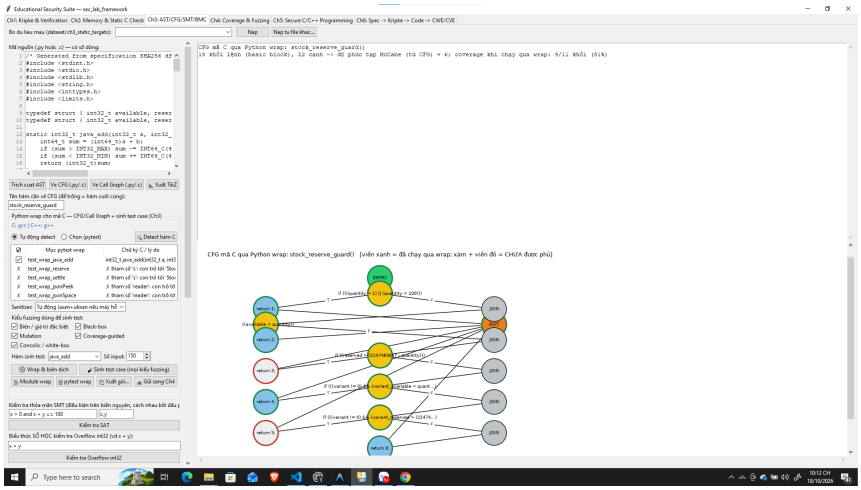
Đối chiếu mục 6.8: GUI ước lượng toàn file McCabe 162, nesting 3 và khoảng 206 câu lệnh. Không nhầm với McCabe của riêng guard hoặc với phép đo Clang.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch3-gui-ast.png

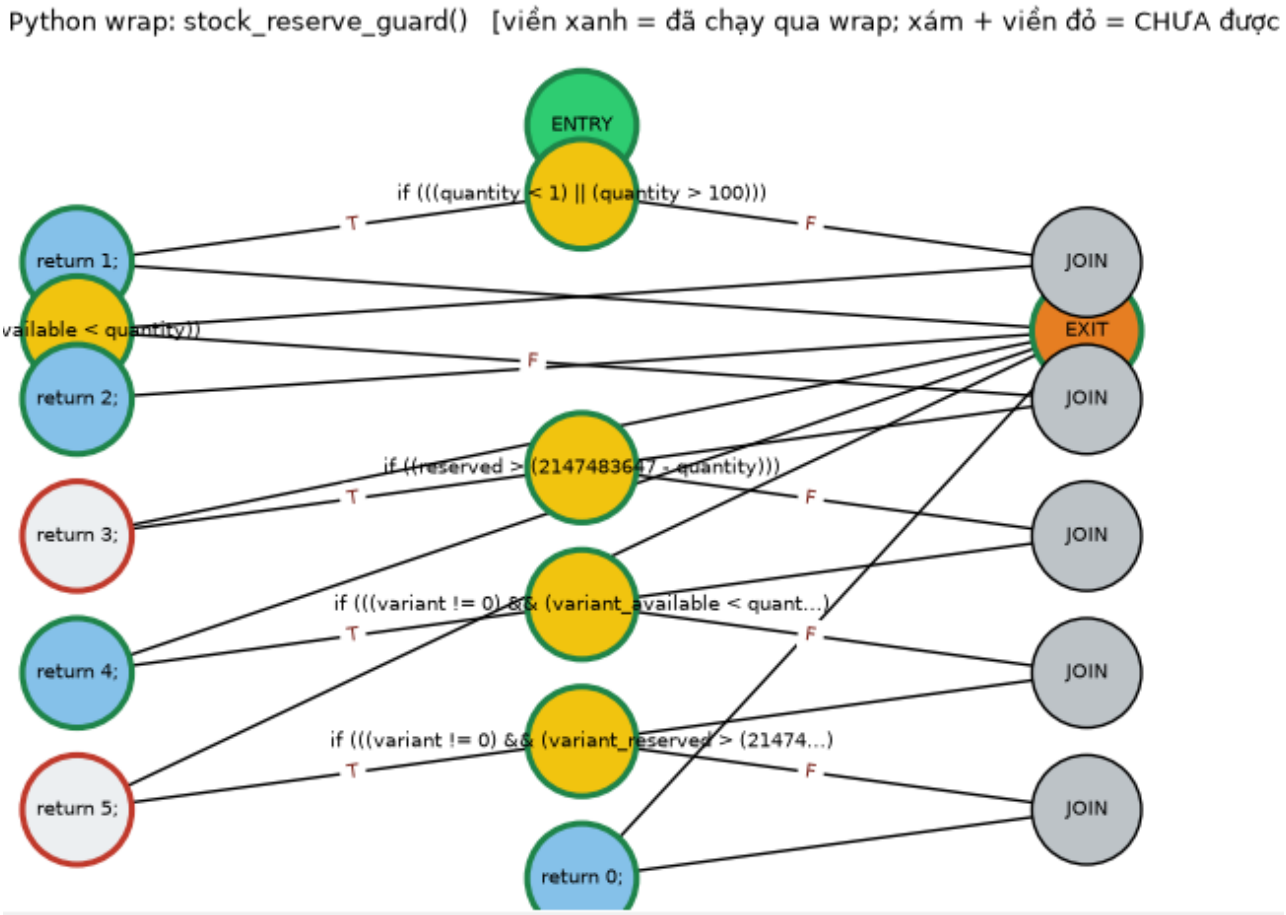
6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.5. Ch3: CFG của stock_reserve_guard

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):



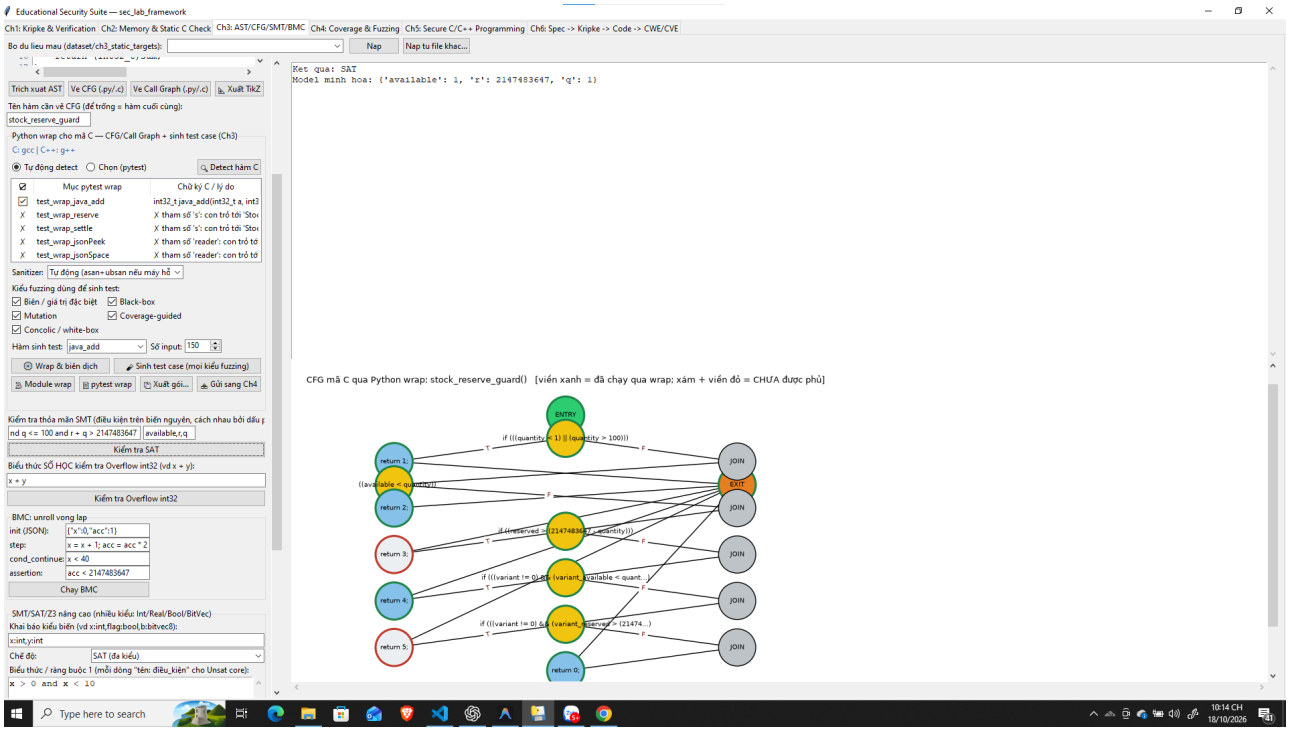
Đối chiếu mục 6.8: ảnh CFG ở lượt wrap ban đầu, 9/11 block (81%), có return 3 và return 5 chưa phủ. Ảnh này không phải lượt concolic 22/22 probe sau đó; hai phép đo không đồng nhất.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch3-gui-cfg.png

6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.6. Ch3: nghiệm SAT của nhánh bị từ chối

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):

```
Ket qua: SAT
Model minh hoa: {'available': 1, 'r': 2147483647, 'q': 1}
```

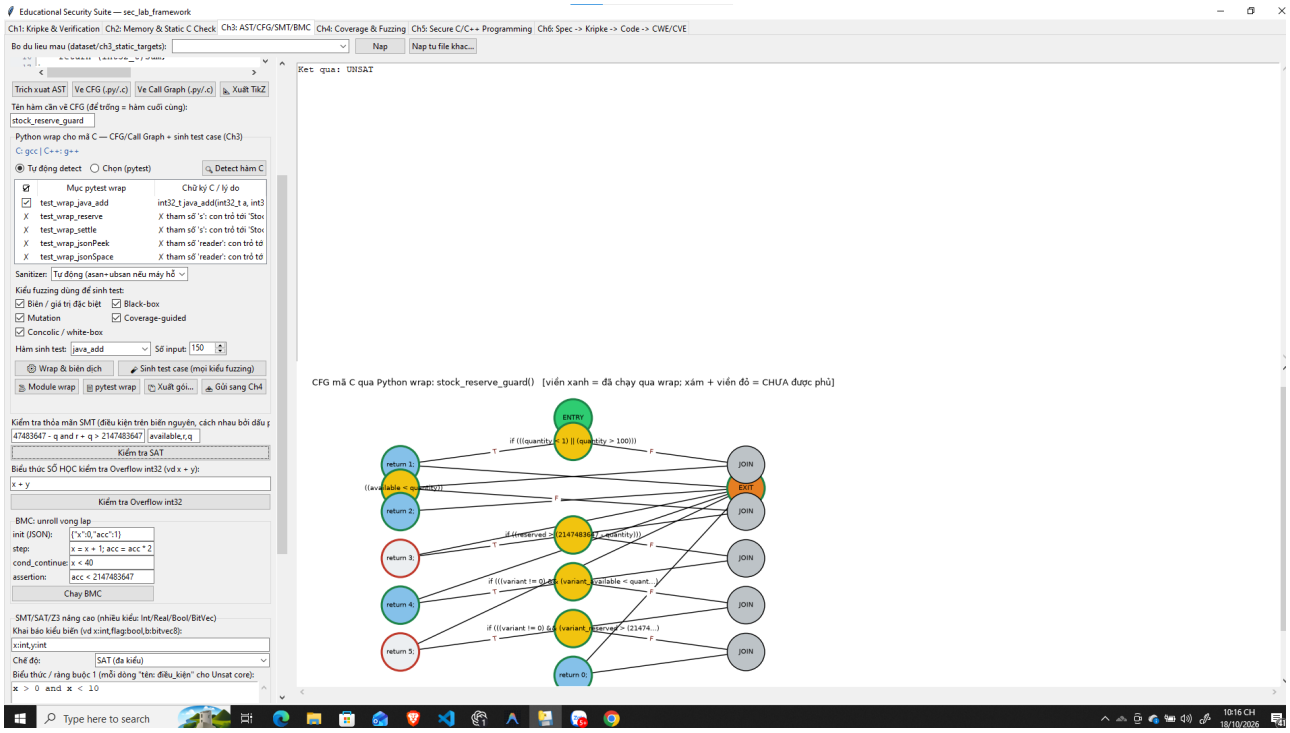
Đối chiếu mục 6.8: console SAT với available=1, r=2147483647, q=1. Đây là điều kiện tìm nhánh guard từ chối tràn, không phải một crash được phát hiện. Công thức được nhập thủ công.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch3-gui-sat.png

6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.7. Ch3: ràng buộc sau guard UNSAT

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):

```
Ket qua: UNSAT
```

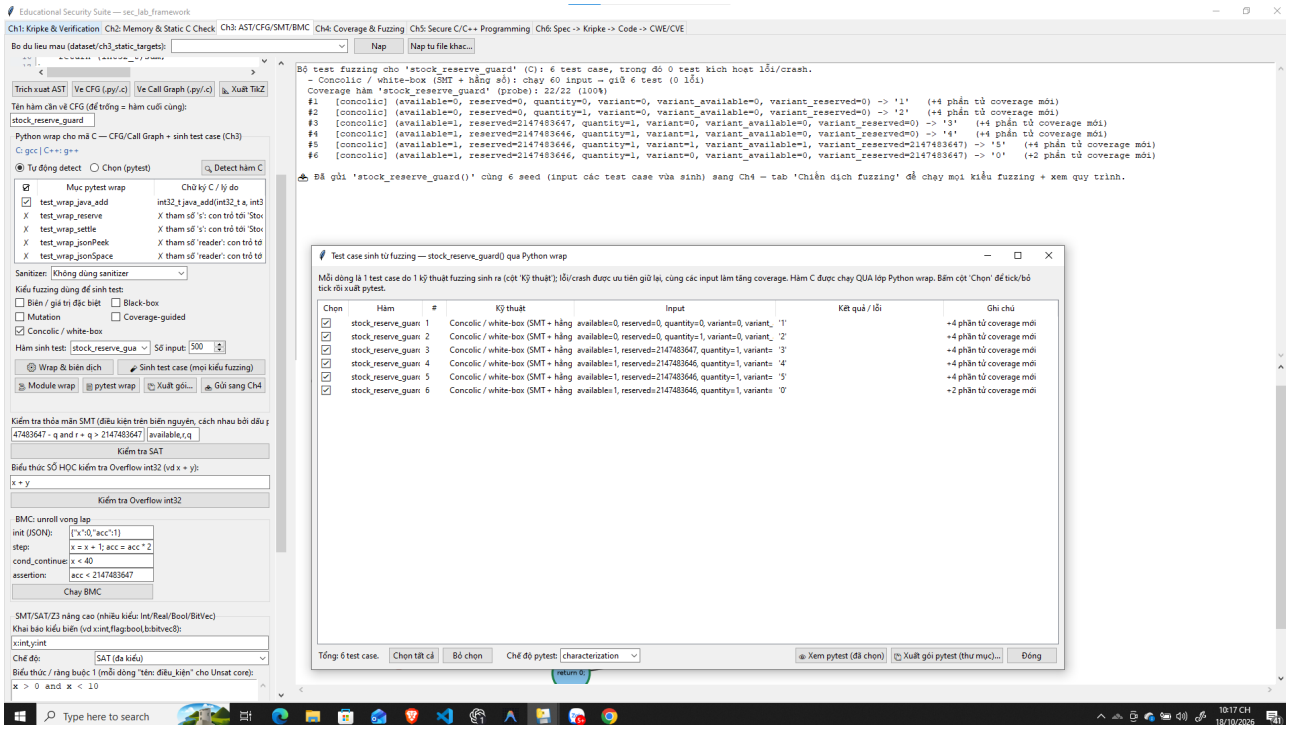
Đối chiếu mục 6.8: console UNSAT khi đồng thời yêu cầu $r \le 2147483647 - q$ và $r + q > 2147483647$. Kết quả cho công thức đã nhập, không phải BMC toàn bộ chương trình.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch3-gui-unsat.png

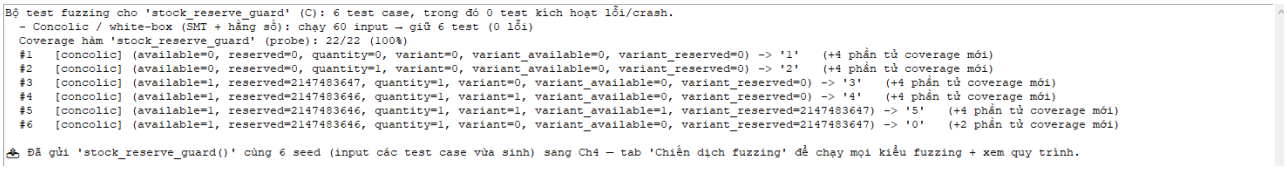
6.10. Ảnh chụp minh chứng (tiếp)

Hình 6.8. Ch3: sinh 6 test từ concolic và chuyển seed

Ảnh màn hình thật do Phong cung cấp ngày 18/10/2026; không dựng lại kết quả.



Chi tiết phóng to từ chính ảnh trên (vùng kết quả):



Đối chiếu mục 6.4: console 60 input, giữ 6 test, 0 crash, 22/22 probe riêng guard; cửa sổ test và thông báo gửi 6 seed sang Ch4. Nút ngân sách 500 không có nghĩa thực tế chạy 500. Ảnh không chứng minh chiến dịch JSON ở Ch4 đã hoàn tất.

Nguồn ảnh: results/ch123-20261018-202400-9a09dbce/ch3-gui-guard-tests.png

6.11. Ảnh thực còn cần bổ sung

Các mục dưới đã có log thực được phân tích nhưng chưa có ảnh chụp màn hình tương ứng để nhúng. Không dùng ảnh vẽ terminal hoặc gõ lại log thành hình để thay thế bằng chứng chạy.

Ảnh cần chụp	Vị trí chèn khi có ảnh	Nội dung phải thấy
Ch4: chiến dịch JSON hoàn tất	Sau 6.1 hoặc 6.3	Tên hàm, seed/ngân sách, số input thực chạy, coverage, số lỗi; đúng lượt/version được mô tả
Terminal: ASan+UBSan	Sau đoạn sanitizer tại 6.6	Lệnh hoặc file log gốc, 41 hồi quy +100 mutation seed 1 và 0 finding; tên thư mục kết quả
Dr. Memory: đối chứng leak và sạch	Sau bảng 6.7	123 byte leak đối chứng bỏ free; đối chứng free đúng không definite leak
Dr. Memory: simulator và potential_errors	Cuối 6.7	1.410 lần gọi, 0 definite leak; potential possible 24 byte ntdll/TLS không bị che
Terminal: pytest gói GUI	Sau bảng 6.4	Lệnh chạy đúng file xuất và dòng 6 passed in 1.65s của log cũ; nếu chạy lại phải ghi thời gian mới
Terminal: so sánh cùng nhánh	Sau bảng 6.9	Nguồn summary đúng thư mục, cả 3 phương pháp, 5 seed và số lần gọi/thời gian

Có thể mở log gốc bằng editor/terminal rồi chụp, nhưng phải chú thích “ảnh xem lại log của lượt ngày 18/10/2026”, không gọi là ảnh chạy trực tiếp. Nếu thực thi lại, lưu bằng chứng và cập nhật số liệu mới, không ghép thời gian của hai lượt.

7. Phản biện & Bài học

Nếu bắt đầu lại, nên thay đổi gì?

Cố định ngay một mục tiêu/nhánh và cách đo trước khi chạy các chiến dịch. Tách hợp đồng parser, nghiệp vụ và guard từ đầu giúp tránh so coverage của các hàm khác nhau. Lưu phiên bản mã nguồn và log gốc của GUI cùng mỗi phép chạy để truy vết thuận tiện hơn.

Điều gì chưa đủ bằng chứng?

Đã có so sánh cùng nhánh và kiểm tra leak bằng Dr. Memory; không còn ghi hai hạng mục này là chưa chạy. Vẫn chưa chạy LeakSanitizer, BMC toàn mô hình hoặc đồng thời/transaction backend. Potential possible leak 24 byte hệ thống chưa được kết luận triệt để. Ch5–Ch6 được tạm gác theo yêu cầu, không ghi hoàn tất.

Bài học phương pháp luận

Cần phân biệt độ phủ với tính đúng, kiểm thử hữu hạn với chứng minh và kiểm tra điều kiện với cập nhật trạng thái. Oracle độc lập và test biến đổi đặc tả giúp phát hiện sai lệch mà test characterization chỉ kiểm tra mã trả về có thể bỏ qua. Không tìm thấy lỗi là một quan sát có ngân sách, không phải kết luận tuyệt đối.

Điều kiện trước khi chốt nộp

• Thông tin nhóm, tên đề tài và mã học phần tạm để trống; điền khi Phong chốt.

• Nếu đăng ký chủ đề có điều kiện riêng như magic/checksum, phải đối chiếu đúng đề tài; phép thử JSON không tự thay thế mọi điều kiện đó.

• Nếu yêu cầu before/after: cần cặp phiên bản và log tái hiện thật, hoặc xác nhận cách trình bày kết quả không tìm thấy lỗi.

• Giữ đúng giới hạn LSan/Dr. Memory và phạm vi mô hình C; Ch5–Ch6 chưa được xác nhận trong bản này.

8. Phụ lục

8.1. Hướng dẫn chạy lại

Mở PowerShell tại C:/Project/Badminton_Stock_Security. Cần Python 3.12+, GCC/gcov cùng phiên bản và LLVM-MinGW cho sanitizer. Dùng .venv đã có trên máy để chạy pytest. Không sửa PATH hệ thống.

```
python -B test_generator.py
```

```
python -B stock_json_fuzz.py --iterations 50 --seeds 1 --seconds 30
```

```
python -B stock_json_fuzz.py --sanitizer --iterations 100 --seeds 1
```

```
.venv/Scripts/python.exe -B -m pytest test_generator.py
```

```
.venv/Scripts/python.exe -B -m pytest test_stock_json_seclab_stock_reserve_guard_fuzz.py
```

Runner có thể tạo thư mục kết quả mới. Bộ compiler portable và .venv không cần đưa vào gói nguồn nộp/GitHub. Không chạy lại builder Excel để thay thế kết quả thực nghiệm.

8.2. Nạp SecLabFramework

Nạp stock_json_seclab.c. Chọn stock_json cho chuỗi JSON, stock_json_case cho fixture số qua parser, hoặc stock_reserve_guard cho concolic thuần số. Biên dịch lại sau khi đổi mã. Giữ đúng nhãn characterization khi xuất pytest. Không chọn helper nhận con trỏ cấu trúc.

8.3. Phân công công việc

Thành viên / MSSV	Công việc	Bằng chứng
.....	.....	.....
.....	.....	.....

8.4. Tài liệu tham khảo

Tài liệu giảng viên: report_guide.pdf (cấu trúc 8 phần, trình bày, before/after); BTL_01_HuongDan_DeTai.pdf (phạm vi, chủ đề 4, yêu cầu bằng chứng và kết quả không tìm thấy lỗi).

MITRE CWE-190: https://cwe.mitre.org/data/definitions/190.html (đối chiếu ngày 18/10/2026).

MITRE CWE-787: https://cwe.mitre.org/data/definitions/787.html (đối chiếu ngày 18/10/2026).

MITRE CWE-476: https://cwe.mitre.org/data/definitions/476.html (đối chiếu ngày 18/10/2026).

MITRE CWE-401: https://cwe.mitre.org/data/definitions/401.html (đối chiếu ngày 18/10/2026).

8.5. Danh mục bằng chứng và hash mã nộp

stock_json_seclab.c

```
dae8dd9162d9e8929f9a67af7cee1555b90ff42ae70995802c0b89ea185a82c5
```

stock_spec.json

```
e078aaccd1b2aa5b2b425c09e4292a6c0cc9488c41d8e042451217efe36bb88c
```

generate_simulator.py

`c5f31423ba6fa0f8e5f117dacbbc1a3cbb8b7c41338b9d74d3ecd03add097bbd`

stock_json_adapter.c

`e559551769049484cb82874ea724b270a260b84a59d000881174e8d1f2b5a915`

8.6. Danh mục đối chiếu bằng chứng

Thư mục / file	Nội dung đối chiếu
results/ch123-20261018-202400-9a09dbce	Ch1 model-check; Ch2 layout/analysis/sanitizer; Ch3 AST/CFG/SMT, ảnh GUI, gói pytest và log
results/comparison-20261018-110108-14dd28b3	summary.json, mã snapshot, miền/ngân sách/thời gian, pytest.xml
results/leak-20261018-225714-fe111d0d	leak-summary.json, cases.json, log đối chứng, simulator, potential_errors
run	Các lệnh chạy lại và thao tác trong ứng dụng; không phải script tự thực thi
badminton_spec.xlsx	10 sheet, 12 yêu cầu, 55 ca tài liệu hóa; không ghi cả 55 ca đã chạy GUI

Các summary và log gốc nằm trong results tương ứng; PDF trình bày số liệu chọn lọc, không chép hàng nghìn dòng log thay cho phân tích. Ảnh GUI do Phong cung cấp, không phải ảnh tác giả báo cáo tự thao tác chụp. Các lượt lịch sử 6.1–6.3 được giữ riêng, không cộng vào một chiến dịch mới.

8.7. Chốt phiên bản và kiểm tra bộ nộp

Chốt phiên bản là chọn đúng bộ C, adapter, generator, spec và nguồn Java tham chiếu đã sinh bằng chứng; ghi hash SHA-256 để phát hiện thay đổi. Không có nghĩa ngừng phát triển website hoặc tự tạo commit/tag. Khi backend thay đổi quy tắc tồn kho, cần đối chiếu lại mô hình; thay đổi giao diện đơn thuần không tự làm sai mô hình đã đóng băng.

Kiểm tra bộ nộp là kiểm tra báo cáo, Excel, mã nguồn, test, file run và log snapshot khớp nhau; đường dẫn/lệnh chạy có đủ phụ thuộc; không đóng gói .env, token, dữ liệu khách hàng, toàn bộ web, .venv hoặc compiler portable. Chưa tạo ZIP hay nộp bài trong lần cập nhật này. Thông tin nhóm vẫn để trống.